

DEPARTEMENT MIASHS - UFR SHS GRENOBLE



Année Universitaire 2025-2026

MÉTHODES DE GÉNIE LOGICIEL : CONCEPTION D'UN SYSTÈME DE GESTION D'ÉVÉNEMENTS

Aylin GUVENTURK
Rachel PERETTI
Louane MAINGUET
Antony TOUMPANOS
Alain KARAPETIAN

Introduction	2
Présentation du projet	3
Analyse des cas d'usage	4
La programmation de festival	4
La billetterie	5
Conception du système	6
Choix de conception et hypothèses	6
Diagramme des classes	6
Nouvelles fonctionnalités	9
Implémentation en Java	9
Implémentation des classes et méthodes	9
Exemples d'instances	10
Conclusions et perspectives	11
Ressources	12

Introduction

Dans le cadre du module de “Méthodes de génie logiciel” du Master MIASHS de l’Université Grenoble Alpes, ce projet porte sur la conception d’un système de gestion d’événements. L’objectif principal est de mettre en pratique les méthodes et outils vu dans le module, notamment la modélisation UML, l’analyse des besoins fonctionnels, ainsi que la justification de choix de conceptions.

Ce rapport présente d’abord les attendus du projet, puis les solutions de conception proposée, ainsi que les hypothèses retenues, à travers différents diagrammes UML et scénarios d’usage. Pour finir, un début d’implémentation des méthodes et classes en Java est présenté, afin d’illustrer notre modélisation; avant de conclure sur notre expérience.

Présentation du projet

Dans le cadre de ce projet, nous avons l'association culturelle MiART souhaitant organiser un festival artistique itinérant qui se déroule dans plusieurs villes. Ce festival comprend différents spectacles (concerts, théâtre de rue, etc...).

Le système à concevoir doit permettre de gérer plusieurs éditions du festival et de réutiliser les entités communes telles que les villes, les lieux, les spectacles et les artistes. Chaque spectacle est caractérisé par un titre, une durée et un ou plusieurs artistes, qui peuvent être des personnes physiques ou des troupes. Lorsqu'un spectacle est programmé dans une ville, des informations spécifiques doivent être renseignées, notamment le lieu, l'horaire, le caractère gratuit ou payant, ainsi que, le cas échéant, le tarif et le nombre de places disponibles.

En complément de la gestion des programmations, le système intègre une fonctionnalité de billetterie. Celle-ci permet aux spectateurs d'acheter les billets en ligne pour un spectacle précis ou des billets couvrant l'ensemble des spectacles d'une ville-étape. On parlera alors de billet-étape.

Enfin, le système doit être capable de prendre en compte des modifications de la programmation, telles qu'un changement d'horaire ou de lieu d'un spectacle. Un mécanisme de notification est alors prévu afin d'informer les composants concernés, notamment la billetterie et les spectateurs ayant acheté un billet.

Le système est destiné aux organisateurs d'événements, qui constituent les utilisateurs principaux. Ils sont chargés de la planification, de la gestion et du suivi du festival.

Analyse des cas d'usage

Nous avons déterminé deux systèmes principaux dans le cadre de la conception pour ce projet.

Un premier qui permet de programmer un festival et un second qui permet de créer et gérer la billetterie. Le second vient se greffer au premier le temps des ventes de billets. Cela nous amène donc à étudier les diverses cas d'usages pour ces systèmes.

La programmation de festival

Pour programmer un festival, seul le ou les organisateur(s) interagissent avec le logiciel. Toute interaction requiert une authentification et une connexion à un compte organisateur. L'organisateur peut réaliser les actions comme décrit dans le diagramme suivant :

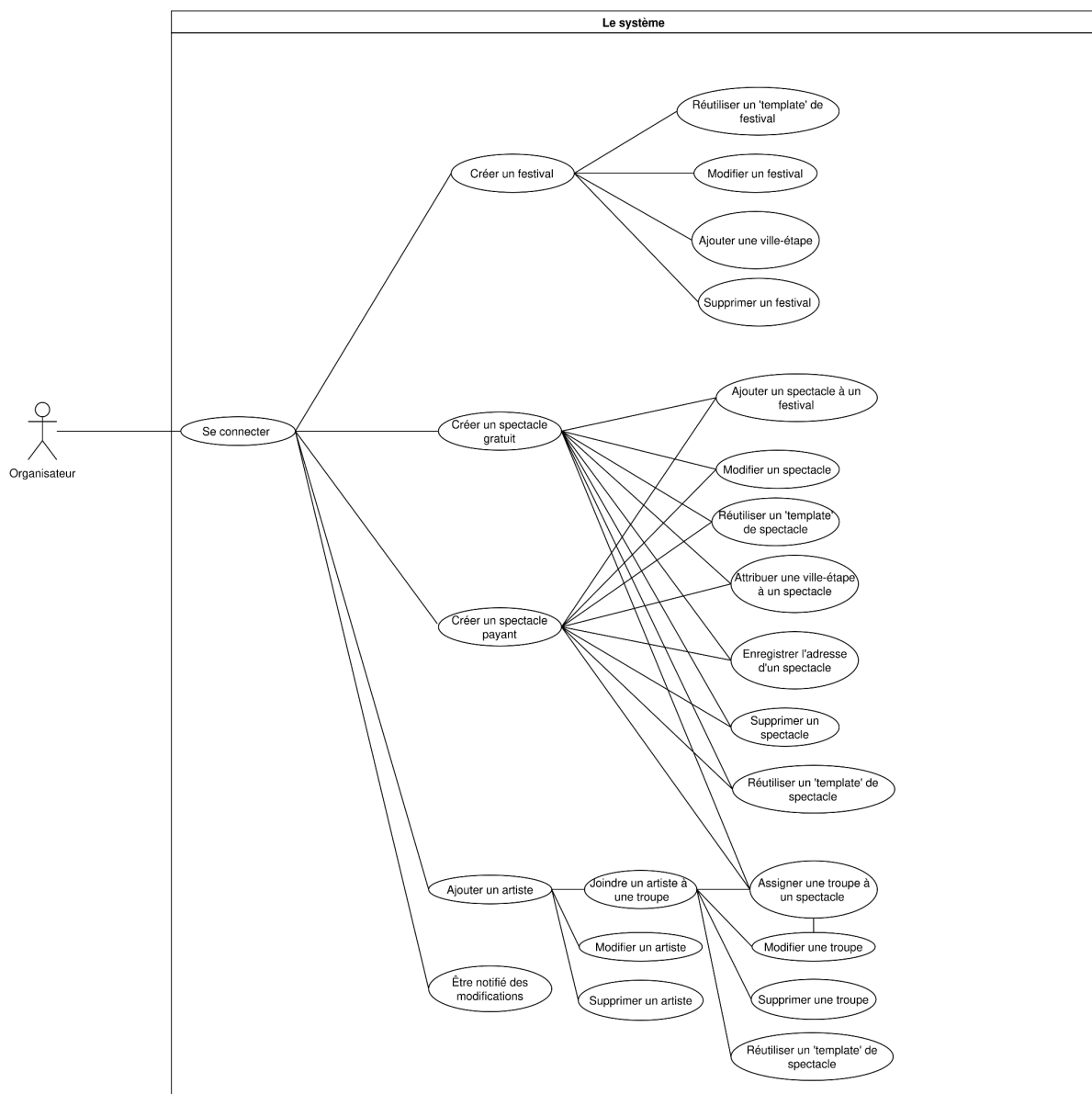


Figure 1. Diagramme des cas d'usage du système Programmation

La billetterie

Pour la billetterie, deux types d'utilisateurs interagissent avec le système: l'organisateur et le spectateur. Tous deux doivent se connecter à un compte dédié afin d'effectuer le rattachement du billet. Le diagramme suivant décrit l'ensemble des interactions que peuvent réaliser les organisateurs et les spectateurs:

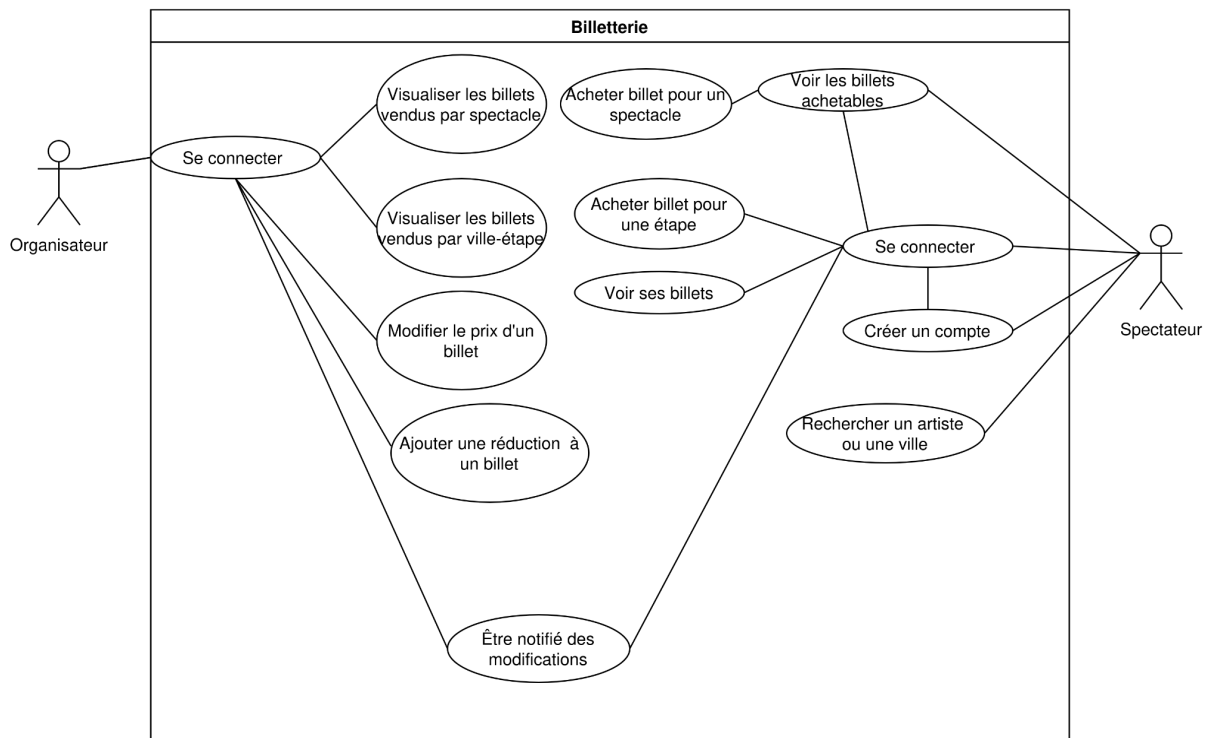


Figure 2. Diagramme des cas d'usage du système Billetterie

Conception du système

Choix de conception et hypothèses

Nous avons fait plusieurs choix importants dans la conception de notre système: au-delà des éléments imposés (festival, spectacle, troupe, etc...), il y a plusieurs séparations qui ont été sujettes à débat.

Tout d'abord pour le lieu, il a été convenu de séparer la salle et la ville au lieu d'avoir un élément "Lieu", afin d'alléger les informations de la salle tout en gardant celles de la ville. Ensuite, pour les spectateurs, il n'y aura pas vraiment de classe "Spectateur" à proprement parler; chaque troupe étant constituée d'artistes, il a été convenu d'avoir un élément "Personne" correspondant à la fois aux spectateurs et aux artistes. Nous considérons le fait qu'un artiste seul est équivalent à une troupe constituée d'une seule personne.

À propos des billets, nous avons créé un élément "Billet" afin de considérer ce qui ne concerne que le billet, et une classe "Billetterie" qui concerne la notion de possession d'un billet et qui comparé à la l'élément "Billet", permet de relier les billets et les personnes.

Pour finir, nous avons décidé pour relier les spectacles, salles et festivals une classe "Programmation" afin de s'occuper de la programmation de spectacles.

Diagramme des classes

Pour permettre aux organisateurs de programmer un festival et aux spectateurs d'acheter des billets, les comportements suivants ont été introduits dans les classes.

Pour les organisateurs, lors de la création du festival, un nom est attribué au festival. Pour chaque programmation de spectacle une instance de `Programmation` est ajoutée à la liste des `Programmation` de la classe `Festival` avec la méthode `addProgrammation()`. Pour consulter l'ensemble de la programmation la méthode `getProgrammation()` renvoie la liste des `Programmation`.

Durant la programmation, quand l'organisateur crée un spectacle, un titre et une durée sont attribués, entrés par l'utilisateur. À l'ajout d'une troupe, la méthode `addTroupe()` ajoute la troupe à la liste de troupes, liste consultable avec `getTroup()`. En créant une troupe le nom est attribué par l'organisateur. L'ajout d'un artiste est exécuté avec la méthode `addArtiste()` en renseignant une instance de la classe `Personne` correspondant à l'artiste.

Ensuite, il faut ajouter dans la programmation du spectacle la salle où performeront les troupes. À la création de la salle, l'organisateur renseigne la ville, le nom de la salle, l'adresse, la capacité, le prix d'une place si elle est payante et la réduction s'il y en a une.

La classe `Salle` servant à stocker les informations relatives à une salle, les méthodes qui sont définies permettent de récupérer ces informations pour la classe `Programmation`. Chaque salle inclut une instance de `Ville`, donc l'utilisateur doit créer une ville en renseignant le nom et le département. La classe `Ville` stocke l'ensemble des programmations dans une liste pour les renseigner à la billetterie. Elle donne accès à la

capacité maximale de la programmation ayant la plus petite capacité de la ville pour la classe billetterie pour proposer des billets étape.

Si l'utilisateur supprime `Spectacle`, `Troupe` déjà créés les méthodes `supprimerSpectacle()`, `supprimerTroupe()` permettront de supprimer les instances de classe respective.

Si l'utilisateur ré-utilise un template de `Festival`, `Spectacle`, `Troupe` déjà créés les méthodes `duplicateFestival()`, `duplicateSpectacle()`, `duplicateTroupe()` permettent de copier coller les instances de classe respective.

Ensuite, la classe `Programmation` permet de relier une salle à un spectacle en stockant l'identifiant du spectacle, la ville, la salle, l'horaire de début, le prix et la capacité de la salle.

Si l'utilisateur modifie l'horaire de début du spectacle, la méthode `modifHoraire()` va mettre à jour l'horaire. La méthode `setCapacite()` permet de réduire la capacité d'une programmation à la vente d'un billet.

Enfin la classe `Festival` permet de relier la billetterie et l'achat des billets aux programmations. A la création de la billetterie, la méthode `createBilletterie()` la crée en renseignant l'ensemble des programmations du festival à la billetterie. La méthode `achatBilletLieu()` permet d'enregistrer la diminution de la capacité d'une salle et d'obtenir le prix de la place. La méthode `achatBilletVille()` permet d'enregistrer la diminution de la capacité de toutes les salles de la ville et d'obtenir le prix total pour l'ensemble des billets avec la méthode `getPriceAllProgramations()`.

Dans le cas des spectateurs, la `Billetterie` permet d'effectuer des achats de billets étapes ou de billets pour un spectacle spécifique. Ces billets seront ensuite stockés dans le compte de la personne grâce à la fonction `getBillet()` de `Personne`.

Les organisateurs, quant à eux, ont la possibilité de visualiser une liste de tous les billets vendus pour une salle ou un festival précis. De plus, la `Billetterie` permet de voir le festival auquel elle est liée et les recettes totales générées celui-ci.

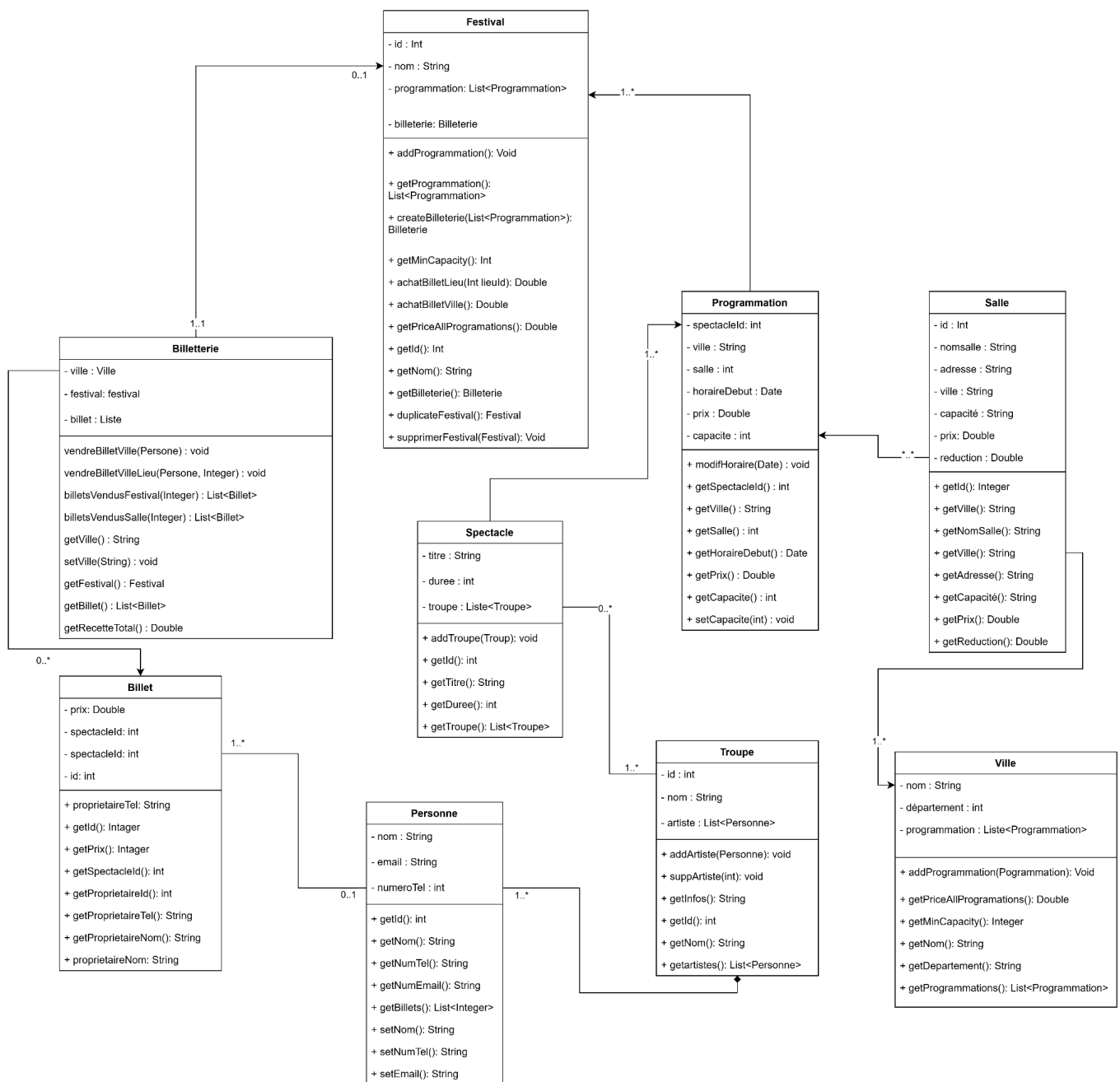


Figure 3. Diagramme de classes du système global

Diagramme de séquence

Le diagramme suivant présente le processus d'achat de billets. Afin d'effectuer l'achat d'un billet, il est nécessaire de commencer par la création d'un compte permettant ensuite de stocker le ou les futurs billets.

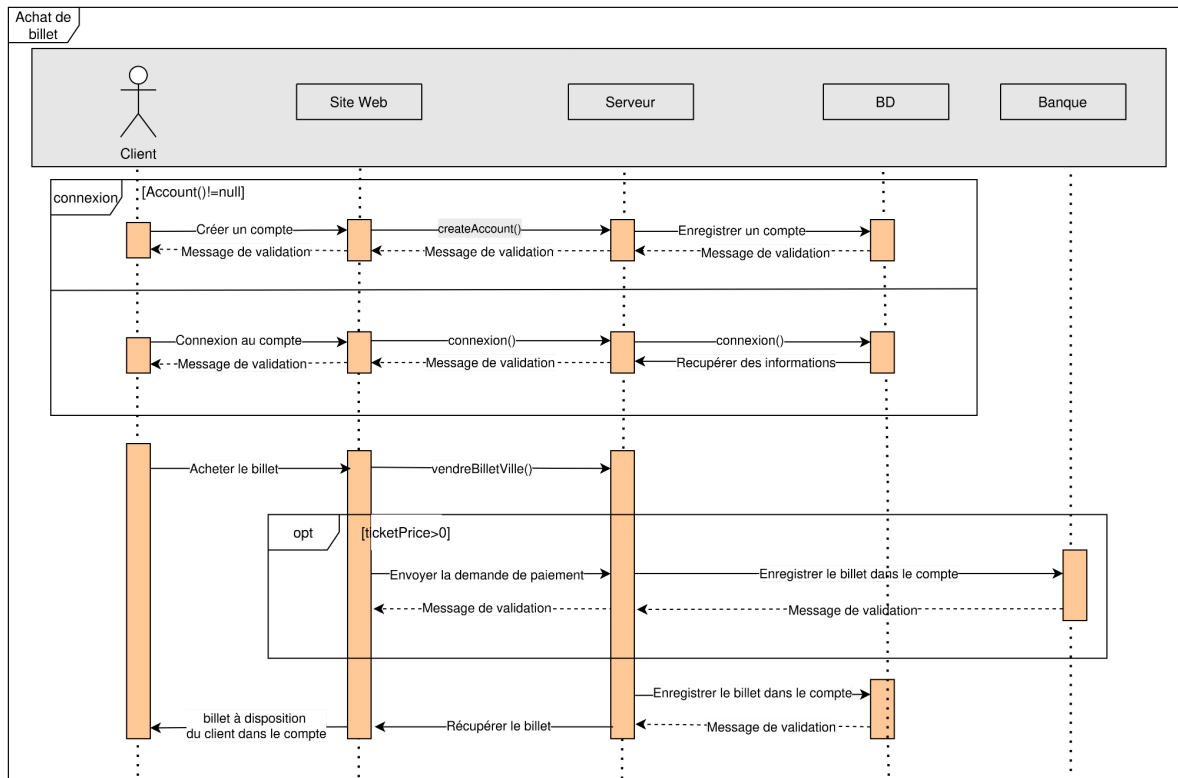


Figure 4. Diagramme de séquence de l'achat de billet étape

Nouvelles fonctionnalités

Nous avons envisagé d'ajouter la possibilité de créer des comptes pour pouvoir stocker les informations personnelles. L'objectif est de pouvoir avoir tous les billets sur un même compte, comme sur la plateforme TicketMaster.

Étant donné que le logiciel est utilisé par les artistes participants aux spectacles, il serait nécessaire d'ajouter des rôles afin de limiter les droits de modification, d'enregistrement et de lecture des informations. Pour cela un attribut devrait être ajouté dans la classe `Personne` afin d'obtenir le statut du compte de la personne et ainsi d'autoriser ou refuser l'accès et la modification de certaines informations. Une autre possibilité serait d'utiliser une librairie Java déjà existante comme JAAS permettant la gestion des rôles.

Implémentation en Java

Implémentation des classes et méthodes

Afin de faire une représentation plus concrète de notre conception, nous avons effectué un début d'implémentation des classes avec leurs méthodes. Toutes les classes sont composées de getters et de setters afin d'améliorer la maintenabilité du logiciel, et de contrôler l'accès aux attributs des classes en évitant leurs manipulations directes, donc offrant une meilleure encapsulation des données. Des fonctions additionnelles permettent de

modifier des attributs lors de changement de programmation ou de renseigner sur des informations de ventes.

Par exemple, la classe `Billetterie` permet de calculer les recettes totales générées par la vente de billets :

```
public Double getRecetteTotal() {
    Double total = 0.0;
    for (Billet billet : billets) {
        total += billet.getPrix();
    }
    return total;
}
```

Exemples d'instances

Suite à l'implémentation, nous avons réalisé une classe `Main` qui effectue la création de plusieurs instances de nos classes, et qui permettent d'afficher les traitements faits sur les festivals, spectacles, et la billetterie.

La figure suivante montre un extrait de la classe `Main` d'instances des classes `Personne`, `Festival` et `Spectacle` :

```
//création personnes
Personne alice = new Personne("Alice", "0600000001", "alice@example.com");
Personne bob = new Personne("Bob", "0600000002", "bob@example.com");
System.out.println("=====");
System.out.println("Personnes créées: " + alice.getNom() + ", " + bob.getNom());

//création festivals
System.out.println("=====");
Festival festivalEte = new Festival("Festival d'été");
Festival festivalAutomne = new Festival("Festival d'automne");
System.out.println("Festival été créé (id=" + festivalEte.getId() + ")");
System.out.println("Festival hiver créé (id=" + festivalAutomne.getId() + ")");

//création spectacles
System.out.println("=====");
Spectacle spectacle1 = new Spectacle("Spectacle Magique", 120);
Spectacle spectacle2 = new Spectacle("Spectacle Fabuleux", 90);
System.out.println("Spectacles créés");
System.out.println("Spectacle Magique créé (id=" + spectacle1.getId() + ")");
System.out.println("Spectacle Fabuleux créé (id=" + spectacle2.getId() + ")");
```

Figure 4. Extrait de la classe `Main`

Conclusions et perspectives

Lors de l'implémentation des classes en Java, une version initiale du projet a d'abord été codée; l'écriture concrète de ce code et des classes prévues nous a permis de nous rendre compte que notre diagramme des classes était incorrect, et nous avons pu par la suite le corriger mais aussi l'améliorer.

Nous avons envisagé de proposer plusieurs modes d'achat de billets, notamment sans création de compte ou en point de vente physique. Pour cela, nous avons pensé à concevoir un logiciel disposant d'une interface dédiée aux employés des guichets, leur permettant d'effectuer les achats de billets pour les spectateurs.

De plus, il serait pertinent d'intégrer l'utilisation de QR codes, afin d'éviter l'impression systématique des billets et de faciliter le contrôle d'accès. Le logiciel devrait donc être multiplateforme, afin de permettre aux visiteurs d'accéder à leurs billets directement depuis leurs smartphones, tout en restant disponible sur ordinateur pour les salariés de l'association. Étant donné que le modèle applicatif est déjà implémenté, il ne resterait plus qu'à développer les vues et les contrôleurs pour rendre le logiciel pleinement utilisable.

Ressources

Lien du code du projet [Github](https://github.com/alkalain/Logiciel-de-Genie/tree/main) : <https://github.com/alkalain/Logiciel-de-Genie/tree/main>